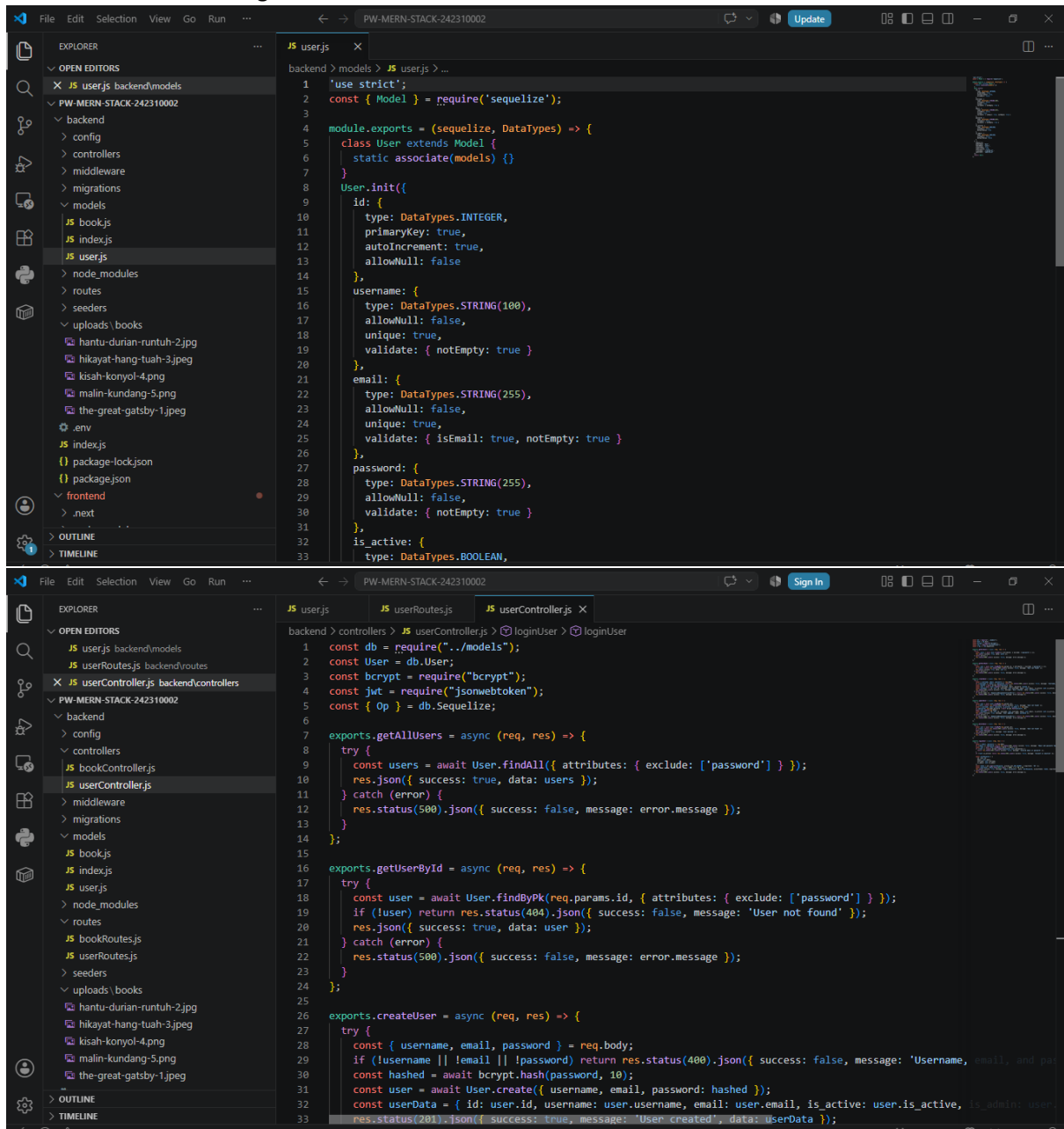


Nama: STEVEN EMMANUEL

Kelas: TI24KA

Npm: 242310002

Mata Kuliah: Lab.PemrogramanWeb



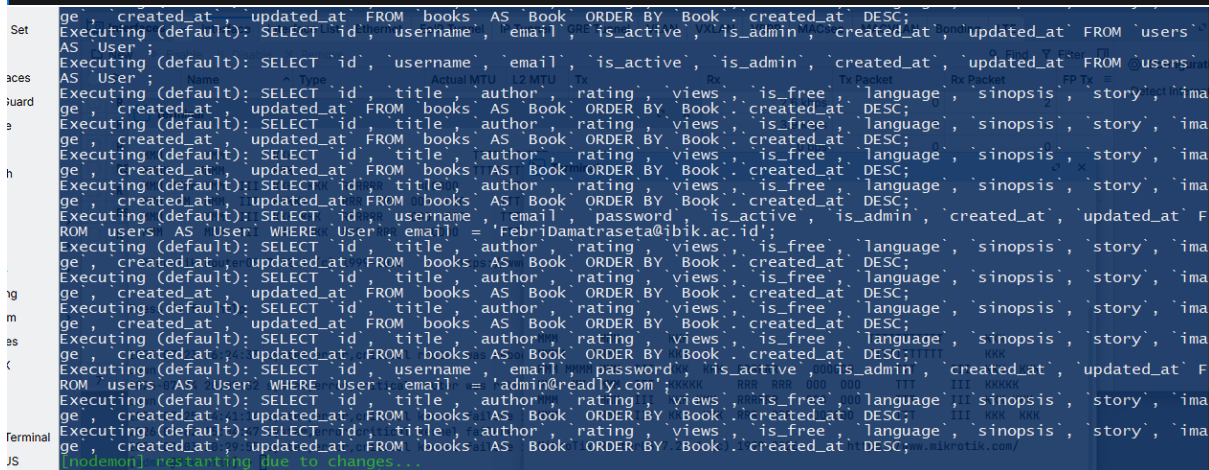
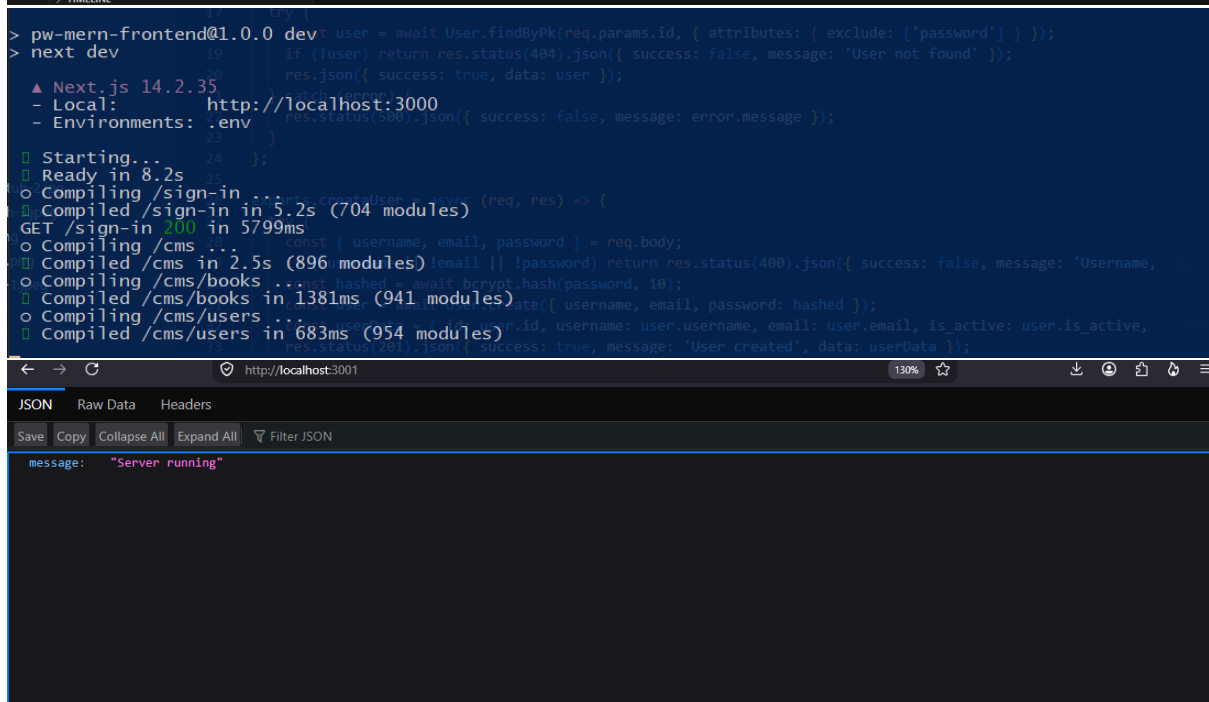
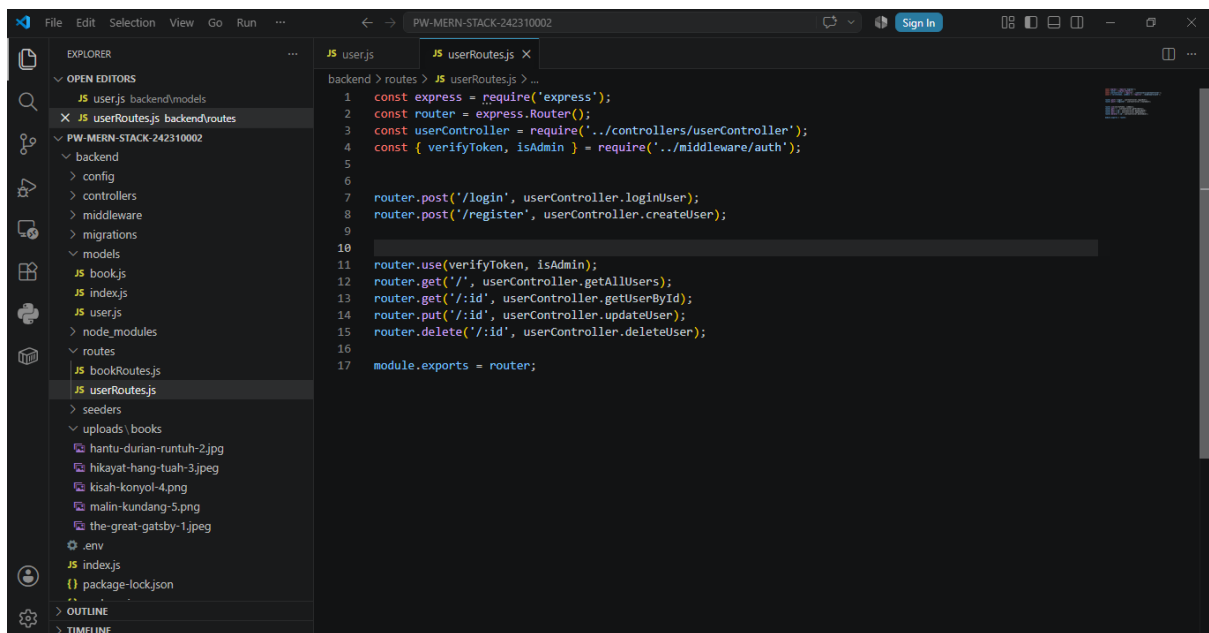
The image displays two screenshots of a Visual Studio Code editor interface, showing the development of a user management API. The top screenshot shows the 'models' directory with the 'user.js' file being edited. The bottom screenshot shows the 'controllers' directory with the 'userController.js' file being edited.

Top Screenshot: models/user.js

```
1 'use strict';
2 const { Model } = require('sequelize');
3
4 module.exports = (sequelize, DataTypes) => {
5   class User extends Model {
6     static associate(models) {}
7   }
8   User.init({
9     id: {
10       type: DataTypes.INTEGER,
11       primaryKey: true,
12       autoIncrement: true,
13       allowNull: false
14     },
15     username: {
16       type: DataTypes.STRING(100),
17       allowNull: false,
18       unique: true,
19       validate: { notEmpty: true }
20     },
21     email: {
22       type: DataTypes.STRING(255),
23       allowNull: false,
24       unique: true,
25       validate: { isEmail: true, notEmpty: true }
26     },
27     password: {
28       type: DataTypes.STRING(255),
29       allowNull: false,
30       validate: { notEmpty: true }
31     },
32     is_active: {
33       type: DataTypes.BOOLEAN,
```

Bottom Screenshot: controllers/userController.js

```
1 const db = require("../models");
2 const User = db.User;
3 const bcrypt = require("bcrypt");
4 const jwt = require("jsonwebtoken");
5 const { Op } = db.Sequelize;
6
7 exports.getAllUsers = async (req, res) => {
8   try {
9     const users = await User.findAll({ attributes: { exclude: ['password'] } });
10    res.json({ success: true, data: users });
11  } catch (error) {
12    res.status(500).json({ success: false, message: error.message });
13  }
14 };
15
16 exports.getUserById = async (req, res) => {
17   try {
18     const user = await User.findByPk(req.params.id, { attributes: { exclude: ['password'] } });
19     if (!user) return res.status(404).json({ success: false, message: 'User not found' });
20     res.json({ success: true, data: user });
21   } catch (error) {
22     res.status(500).json({ success: false, message: error.message });
23   }
24 };
25
26 exports.createUser = async (req, res) => {
27   try {
28     const { username, email, password } = req.body;
29     if (!username || !email || !password) return res.status(400).json({ success: false, message: 'Username, email, and password are required' });
30     const hashed = await bcrypt.hash(password, 10);
31     const user = await User.create({ username, email, password: hashed });
32     const userData = { id: user.id, username: user.username, email: user.email, is_active: user.is_active, is_admin: user.is_admin };
33     res.status(201).json({ success: true, message: 'User created', data: userData });
34   } catch (error) {
35     res.status(500).json({ success: false, message: error.message });
36   }
37 };
```



Operasi CRUD (Dilindungi)

GET /api/users – Mengambil semua pengguna (gak termasuk kata sandi).

GET /api/users/:id – Mengambil satu pengguna berdasarkan ID.

POST /api/users – Membuat pengguna baru (admin).

PUT /api/users/:id – Memperbarui pengguna secara penuh (admin). kata sandi baru di-hash, kata sandi tersebut akan di-hash.

DELETE /api/users/:id – Menghapus seorang pengguna (admin).